

Data Analysis and Visualization

PART II:

The Art of Summarizing Data and Visualization

Lecturer: Diu Tran, PhD

Department of Computer Science
Faculty of Information Technology
Ton Duc Thang University

Ho Chi Minh, Week 2, 2025

Content

- 1 Revise last week
- 2 Exploring and pre-processing datasets
- 3 Share data through the Art of Visualization
- 4 Types of plots
 - Line plots
 - Area plots
 - Histograms
 - Bar plots
 - Pie charts
 - Box plots
 - Scatter plots
 - Bubble Plots
- 5 Lab 2

Revise last week

Data analysis process to "Learning from data" (with examples):

- ➊ Define the problem, research of interest: the transportation department wants to increase the use of buses within the city.
- ➋ Collecting the data:
 - ▶ Specify the objective of the study: ask effective questions in order to find which datasets should be collected such as which factors can increase the number of people using bus.
 - ▶ Identify the variables of interest: e.g., identify factors such as safety, cost, cleanliness of the buses, whether or not there is a bus stop close to the person's home or place of employment, how often the bus fails to be on time, demographic information, place of residence, how frequently a person currently rides buses
 - ▶ Choose methods to collect data: surveys, experiments, and the examination of existing data from business records, censuses, government records, and previous studies
 - ▶ Collecting data

ChatGPT assists in brainstorming

give an example of a research of interest, then ask effective question in order to find which data should be collected, identify particular variables from the effective questions so that we can find suitable resources to collect datasets for our investigate

Absolutely! Here's a full example that goes through all the steps you're asking for — including the **research problem**, **effective guiding questions**, and **identifying variables** to guide dataset collection.

Example Research of Interest:

"What is the impact of social media usage on the mental health of university students?"

Step 1: Ask Effective Questions to Determine What Data to Collect

1. What defines "mental health" in this context?
 - Are we focusing on stress levels, anxiety, depression, sleep quality, or general well-being?
2. How do we measure social media usage?
 - Time spent per day? Number of platforms used? Type of content consumed (e.g., academic vs. entertainment)?
3. Are we considering demographic factors?
 - Does gender, age, year of study, or field of study affect the relationship?
4. Is the impact short-term or long-term?
 - Are we measuring mental health outcomes over a semester, academic year, or daily logs?
5. Are there mediating factors?
 - Do factors like physical activity, academic pressure, or in-person social interaction mediate the effect?



Step 2: Identify Variables from the Questions

Category	Variable Name	Type
Mental Health	Depression score (e.g., PHQ-9)	Continuous
	Anxiety score (e.g., GAD-7)	Continuous
	Sleep quality	Ordinal / Score
	Stress level	Continuous
Social Media Usage	Time on social media per day	Continuous
	Number of platforms used	Discrete
	Type of content consumed	Categorical
Demographics	Age	Continuous
	Gender	Categorical
	Year of study	Ordinal
	Major / Program	Categorical
Mediating Variables	Physical activity (e.g., hours/week)	Continuous
	Academic workload (e.g., credits taken)	Continuous
	Social support scale	Score (ordinal)

Step 3: Find Suitable Data Sources / Datasets

Here's where you might look for data:

1. Surveys and Questionnaires:

- Use standardized tools like PHQ-9 (for depression), GAD-7 (for anxiety), and Pittsburgh Sleep Quality Index.
- You can collect these yourself via surveys or find datasets that already include them (e.g., from university health studies).

2. Digital Behavior Tracking:

- Some studies (like those on Kaggle or in academic repositories) track digital behavior, including social media time via apps or self-report.

3. University Health Research:

- Look for open datasets from institutions like:
 - Harvard Dataverse
 - ICPSR (Inter-university Consortium for Political and Social Research)
 - Kaggle (search "student mental health")
 - WHO or CDC mental health surveys for students

4. Custom Data Collection:

- If you can't find a dataset that meets all needs, design a survey using Google Forms or Qualtrics targeting university students, collecting all the variables identified.

Summary

- **Research Problem:** How does social media use impact mental health among university students?
- **Key Variables:**
 - Independent: Time on social media, type of content, platforms used
 - Dependent: Mental health scores (e.g., depression, anxiety, sleep)
 - Control/Moderator: Age, gender, academic workload, physical activity
- **Data Sources:**
 - Academic repositories, survey platforms, mental health screening tools, or direct collection via student outreach



Exploring and pre-processing data using Pandas

Preprocessing_Exploring_with_Pandas.ipynb

Skills:

- Read, view, save dataset
- Uncover some information of data: data shape, column names, indices, data types
- Check null values
- Calculate new features which can be used for investigating
- Extract statistical information of the dataset
- Select columns, select rows
- Filter the dataframe based on a condition
- Sorting Values of a dataframe

Cheat Sheet : Data Preprocessing Tasks in Pandas

Task	Syntax	Description	Example
Load CSV data	<code>pd.read_csv('filename.csv')</code>	Read data from a CSV file into a Pandas DataFrame	<code>df_can=pd.read_csv('data.csv')</code>
Handling Missing Values	<code>df.dropna()</code>	Drop rows with missing values	<code>df_can.dropna()</code>
	<code>df.fillna(value)</code>	Fill missing values with a specified value	<code>df_can.fillna(0)</code>
Removing Duplicates	<code>df.drop_duplicates()</code>	Remove duplicate rows	<code>df_can.drop_duplicates()</code>
Renaming Columns	<code>df.rename(columns={'old_name': 'new_name'})</code>	Rename one or more columns	<code>df_can.rename(columns={'Age': 'Years'})</code>
Selecting Columns	<code>df['column_name']</code> or <code>df.column_name</code>	Select a single column	<code>df_can.Age</code> or <code>df_can['Age']</code>
	<code>df[['col1', 'col2']]</code>	Select multiple columns	<code>df_can[['Name', 'Age']]</code>
Filtering Rows	<code>df[df['column'] > value]</code>	Filter rows based on a condition	<code>df_can[df_can['Age'] > 30]</code>
Applying Functions to Columns	<code>df['column'].apply(function_name)</code>	Apply a function to transform values in a column	<code>df_can['Age'].apply(lambda x: x + 1)</code>
Creating New Columns	<code>df['new_column'] = expression</code>	Create a new column with values derived from existing ones	<code>df_can['Total'] = df_can['Quantity'] * df_can['Price']</code>
Grouping and Aggregating	<code>df.groupby('column').agg({'col1': 'sum', 'col2': 'mean'})</code>	Group rows by a column and apply aggregate functions	<code>df_can.groupby('Category').agg({'Total': 'mean'})</code>
Sorting Rows	<code>df.sort_values('column', ascending=True/False)</code>	Sort rows based on a column	<code>df_can.sort_values('Date', ascending=True)</code>
Displaying First n Rows	<code>df.head(n)</code>	Show the first n rows of the DataFrame	<code>df_can.head(3)</code>
Displaying Last n Rows	<code>df.tail(n)</code>	Show the last n rows of the DataFrame	<code>df_can.tail(3)</code>
Checking for Null Values	<code>df.isnull()</code>	Check for null values in the DataFrame	<code>df_can.isnull()</code>
Selecting Rows by Index	<code>df.iloc[index]</code>	Select rows based on integer index	<code>df_can.iloc[3]</code>
	<code>df.iloc[start:end]</code>	Select rows in a specified range	<code>df_can.iloc[2:5]</code>
Selecting Rows by Label	<code>df.loc[label]</code>	Select rows based on label/index name	<code>df_can.loc['Label']</code>
	<code>df.loc[start:end]</code>	Select rows in a specified label/index range	<code>df_can.loc['Age':'Quantity']</code>
Summary Statistics	<code>df.describe()</code>	Generates descriptive statistics for numerical columns	<code>df_can.describe()</code>

The art of data visualization

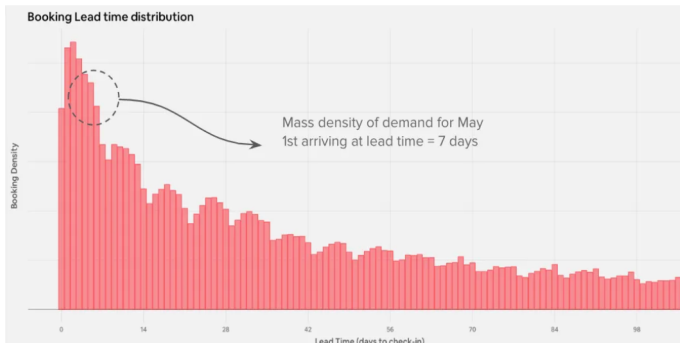
What is data visualization?

- Data visualization is the graphical representation of data and information. It involves the process of creating visual representations of data.
- Data visualization can take many different forms, from basic charts and graphs to more complex interactive dashboards, maps, and infographics.
- Basic charts and graphs are the simplest form of data visualization to represent numerical data.
- Interactive dashboards and maps are more complex forms of visualization to provide real-time information.

Why build visuals?

- Easily understand data: highlight patterns, trends, and relationships that might not be immediately apparent from looking at the data
- Communicate insights: simplifies the data for stakeholders in a more compelling and understandable way
- Identify trends and data patterns that might be difficult to see: help gain new insights and make informed decisions by examining data over time or across different dimensions
- Present data for better understanding and interpretation: help us to identify opportunities, spot potential problems, and make more informed choices

The power of data visualization

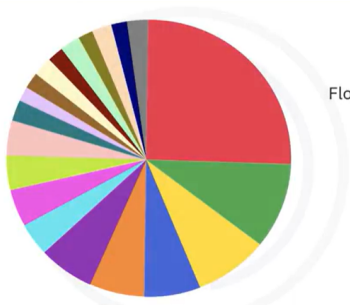


Source: <https://medium.com/airbnb-engineering/learning-market-dynamics-for-optimal-pricing-97cffbcc53e3>

Bar chart displays the lead time distribution aggregated to a market level. From which, assist hosts and determining the best possible prices for their listings, thereby increasing their profits.

The power of data visualization

Spotify Pie
Bake Your Monthly Genre Pie
[GO TO SPOTIFY](#)
[LOGIN TO SPOTIFY](#)

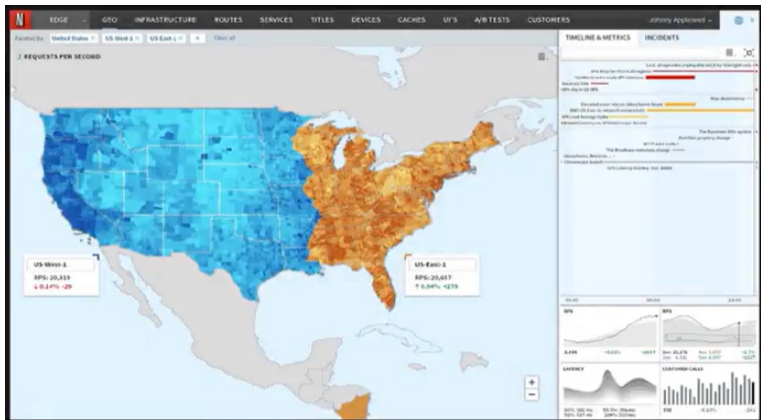


Taylor Swift
Harry Styles
Florence + The Machine
Phoebe Bridgers
One Direction
Lorde
Charli XCX
Niall Horan
Jack Harlow
ASAP Rocky
Kacey Musgraves
Tyler, The Creator
Hozier
Mitski
Omar Apollo
5 Seconds of Summer

Spotify uses Spotify Pie, a unique data visualization tool that enables users to create customized pie charts based on their listening habits on Spotify. The chart displays the distribution of different genres, artists, and tracks the user has listened to.

Pie charts displays the distribution of different genres, artists, and tracks the user has listened to.

The power of data visualization



A choropleth map with other visualizations like line graphs and bar plots help gain a deeper understanding of operations, identify areas for improvement, and quickly address any issues.

The power of data visualization

Data visualization is an essential tool for anyone dealing with data

- Business and Finance: gain insights into market trends, financial performance, customer behavior,
- Healthcare: identify patterns in patient data and develop targeted treatment plans,
- Education: analyze student performance and inform instructional decisions leading to better academic outcomes,
- Government: help governments make informed decisions and communicate data to the public,
- Research and Science: analyze complex data, develop new insights, and share findings effectively,
- Entertainment: determine ratings and ultimately signals to the creators what the audience wants, ...
- etc, ...

Data visualization

- Represent data using visual formats
- Effectively communicate information, trends and insights
- Uncover patterns, identify trends
- Present complex information in an easier way to understand

Types of plots

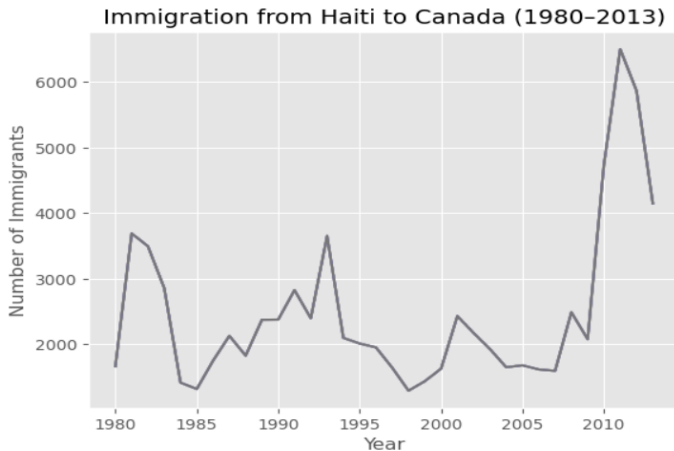
- Line plots
- Area plots
- Histograms
- Bar plots
- Pie charts
- Box plots
- Scatter plots
- Bubble plots
- Treemaps
- Pivot charts
- ...

Files:

- `Data_visualization1_Matplotlib.ipynb`
- `Data_visualization2_Matplotlib.ipynb`

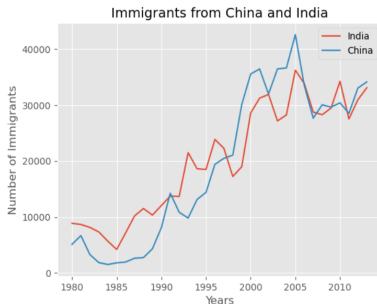
Line plots

A line plot, also known as a line chart, displays data as a series of data points connected by straight lines.



Line plots: use cases

- Display trends over time: stock market fluctuations, temperature changes over time
- Compare datasets with a continuous independent variable: age or time
- Illustrate cause and effect relationships: sales revenue changes based on marketing budget
- Visualize continuous data: stock prices over time, height measurements over time

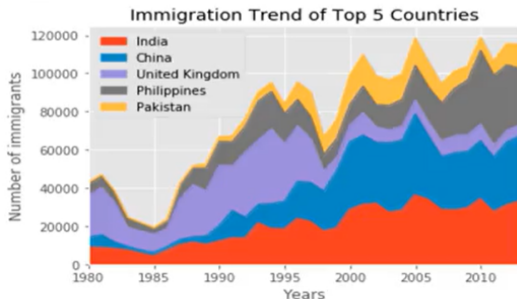


Area plots

- Also known as an area chart or graph
- Displays the magnitude and proportion of multiple variables
- Represents time or another ordered dimension
- Is based on the line plot

Area plots: use cases

- Tracking stock market performance
- Visualizing population demographics
- Displaying the distribution of resources across various sectors

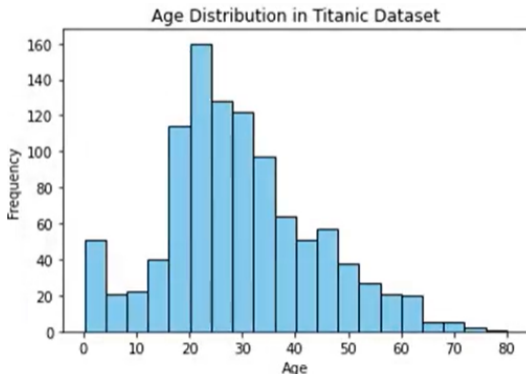


Jupyter notebook - code demo

- An area plot depicts cumulated totals using numbers or percentages over time
- The process of creating an area plot involves importing Matplotlib and calling the plot function on the dataframe with kind parameter assigned as area.
- Area plots provide a visually appealing and intuitive way to showcase the relationship and proportion of multiple variables in a single chart.

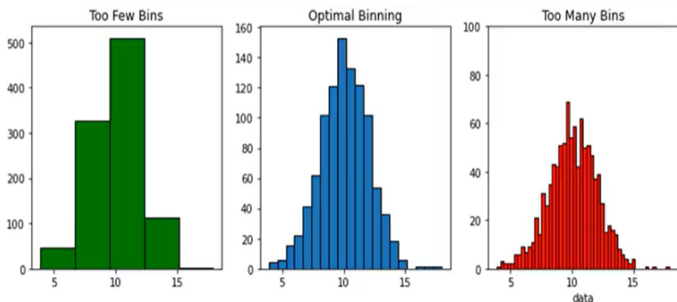
Histograms

- Graphical representation of dataset distribution
- Show the frequency or relative frequency of values within intervals
- Bars represent the data count in each interval



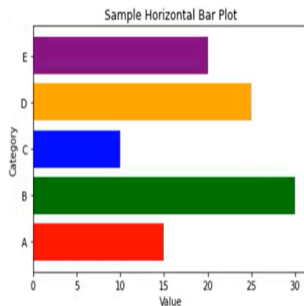
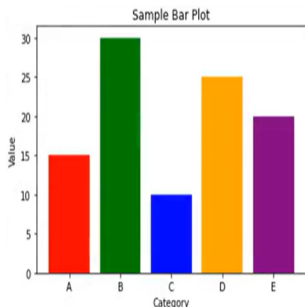
Histograms: use cases

- Understand data distribution
- Visually depict the shape of the data
- Assess skewness in the data
- Showcase data variability and spread



Bar plots

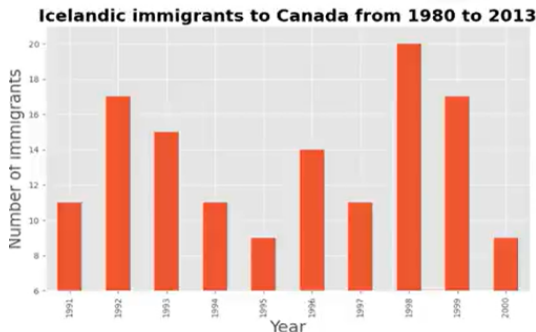
- Displays data using rectangular bars
- The height or length of the bars represents the magnitude of the data
- Displays bars either vertically or horizontally



Bar plots: use cases

- Compare different categories or groups
- Display discrete data that has distinct categories
- Show how different categories contribute to the whole
- Visualize data that you can easily rank or order

Example:

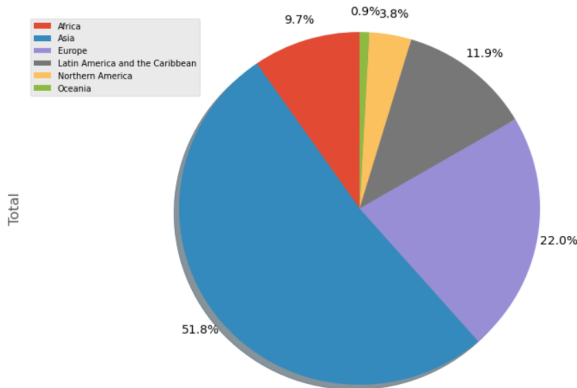


Pie charts

A pie chart is a circular statistical graphic divided into segments to illustrate numerical proportion.

Jupyter notebook: `Data_visualization2_Matplotlib.ipynb`

Immigration to Canada by Continent [1980 - 2013]

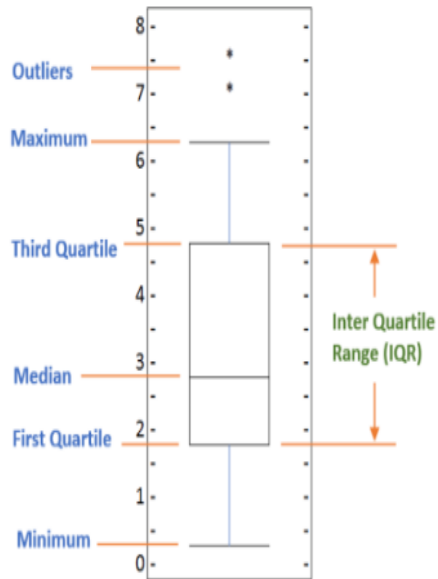


Box plots

- Displays the distribution of a dataset along with key statistical measures
- Represents the interquartile range (IQR) as a box, a median line and whiskers indicating the range
- Represents outliers as individual data points beyond whiskers
- A box plot is a way of statistically representing given data distribution through five main dimensions.
- The five main dimensions are minimum, first quartile, median, third quartile, and maximum.

Jupyter notebook: `Data_visualization2_Matplotlib.ipynb`

Box plots



Box plots: statistical information

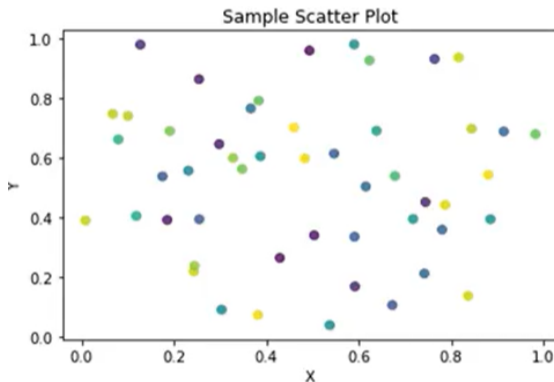
- Minimum is usually the value obtained by subtracting 1.5 times the interquartile range, or IQR, from the first quartile
- First quartile is the point 25 % of the way through the sorted data. In other words, a quarter of the data points are less than this value.
- Median is the median of the sorted data
- The third quartile is the point 75 % of the way through the sorted data. In other words, three quarters of the data points are less than this value
- The maximum is usually the value obtained by adding the third quartile to 1.5 times the IQR
- Box plots also display outliers as individual dots outside the outer and lower extremes

Box plots: use cases

- Compare the distribution of a continuous variable across different categories or groups: e.g., employees salaries across departments
- Examine spread and skewness of dataset, visualizing quartiles
- Identify and analyze potential outliers
- Visualize summary statistics
- Compare distributions of multiple variables in datasets

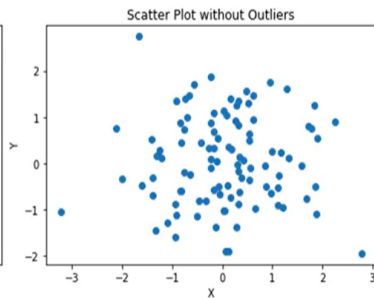
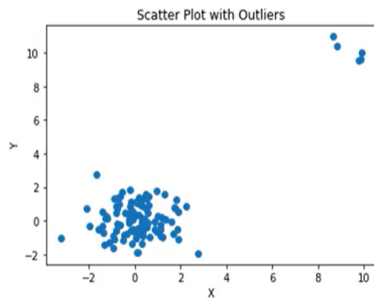
Scatter plots

- Use Cartesian coordinates to display values for two variables
- Value of one variable determines the position on the horizontal axis and the value of other variable determines the position on the vertical axis
- Determine the correlation between the two variables



Scatter plots: use cases

- Examine the relationship between two continuous variables
- Investigating patterns or trends in data
- Detect outliers or unusual observations
- Identify clusters or groups in the data
- Explore complex data



Bubble Plots

Data_visualization2_Matplotlib.ipynb

Summarize types of plots in data visualization

Data visualization is the process of presenting data in a visual format, such as charts, graphs, and maps, to help people understand and analyze data easily.

- Line plots capture trends and changes over time, allowing us to see patterns and fluctuations
- Bar plots compare categories or groups, providing a visual comparison of their values
- Scatter plots explore relationships between variables, helping us identify correlations or trends
- Box plots display the distribution of data, showcasing the median, quartiles and outliers
- Histograms illustrate the distribution of data within specific intervals, allowing us to understand its shape and concentration

Plot libraries in Python

- Matplotlib: a plotting library that offers a wide range of plotting capabilities, one of the most widely used data visualization libraries in Python, a well-established data visualization library that can be integrated in different environments.
- Pandas: fundamentally used for data manipulation but also has plotting functionality
- Seaborn: statistical data visualization
- Folium: geospatial data visualization
- Plotly: an interactive and dynamic library for data visualization that supports a wide range of plot types and interactive features.
- PyWaffle: plotting waffle charts

Examples: Jupyter Notebook

Cheat Sheet : Plot Libraries

Library	Main Purpose	Key Features	Programming Language	Level of Customization	Dashboard Capabilities	Types of Plots Possible
Matplotlib	General-purpose plotting	Comprehensive plot types and variety of customization options	Python	High	Requires additional components and customization	Line plots, scatter plots, bar charts, histograms, pie charts, box plots, heatmaps, etc.
Pandas	Fundamentally used for data manipulation but also has plotting functionality	Easy to plot directly on Panda data structures	Python	Medium	Can be combined with web frameworks for creating dashboards	Line plots, scatter plots, bar charts, histograms, pie charts, box plots, etc.
Seaborn	Statistical data visualization	Stylish, specialized statistical plot types	Python	Medium	Can be combined with other libraries to display plots on dashboards	Heatmaps, violin plots, scatter plots, bar plots, count plots, etc.
Plotly	Interactive data visualization	interactive web-based visualizations	Python, R, JavaScript	High	Dash framework is dedicated for building interactive dashboards	Line plots, scatter plots, bar charts, pie charts, 3D plots, choropleth maps, etc.
Folium	Geospatial data visualization	Interactive, customizable maps	Python	Medium	For incorporating maps into dashboards, it can be integrated with other frameworks/libraries	Choropleth maps, point maps, heatmaps, etc.
PyWaffle	Plotting Waffle charts	Waffle charts	Python	Low	Can be combined with other libraries to display waffle chart on dashboards	Waffle charts, square pie charts, donut charts, etc.

Lab 2

Using the datasets that you collected from Lab 1:

- 1 Practice wrangle data with Pandas: data pre-processing, data summarizing. Reference
`Preprocessing_Exploring_with_Pandas.ipynb`
- 2 Data visualization using Matplotlib. Reference
`Data_visualization1_Matplotlib.ipynb` and
`Data_visualization2_Matplotlib.ipynb`